

Cascade Chat Conversation

First, we will provide an analysis of the chat conversation and then analyze the code that the Windsurf LLM produced

Note: This is purely the output of the chat conversation and does not contain any raw data, codebase snippets, etc. used to generate the output.

User Input

You are my pair-programmer inside Windsurf. I have two notebooks with TODOs: q_coding_ssm_forward_cpu.ipynb q_coding_ssm_forward_gpu.ipynb The problem is implementing the SSM forward pass for: $h_{t+1} = W h_t + U x_t + b h_{t+1} = W h_t + U x_t + b$ Given all inputs $x_{t,txt}$ for $t=1..T$ and $h_0=0$, compute all hidden states $h_1..h_T$. Your task Implement every TODO exactly as the notebooks intend, with correct tensor shapes, correctness checks, and runtime comparisons. Hard requirements (do not violate) Do not change any provided function signatures, cell order, or variable names unless a TODO explicitly asks for refactoring. Fill in TODO blocks only. Use PyTorch only (plus standard Python utilities like time, torch.utils.benchmark if allowed). Make results numerically correct and match the recurrence. Make runtime comparisons fair: warmup, synchronize CUDA when timing, same dtype/device, same shapes.

What to implement CPU notebook (a) Recurrent unroll Implement a function that computes all states by looping over time. Use efficient tensor ops inside the loop (matmul/addmm), avoid Python overhead where possible but correctness first. Output shape must match the notebook's expected format (likely [T, H] or [B, T, H]). (b) Convolution-based approach Implement the forward using a 1D convolution idea: Expand the recurrence: $h_t = \sum_{k=0}^{t-1} W^{t-k} U x_k + b$ Construct a kernel sequence $K[\ell] = W^\ell U$ for $\ell=0..T-1$ and apply it over time. Also incorporate the bias contribution: $\sum_{\ell=0}^{t-1} W^\ell b$ (either as a separate precomputed sequence added after conv, or folded into the "input" by augmenting $x_{t,txt}$ with a constant 1 and augmenting UUU accordingly if the notebook allows). Use `nn.Conv1d` correctly: Decide the expected tensor layout (`Conv1d` expects [N, C_in, L]). Ensure kernel weights have shape [C_out, C_in, K]. Be careful about causal alignment: you likely need padding and/or flipping the kernel in time depending on how you map indices. Important: The hint says "create the kernel using a divide-and-conquer method." Implement a fast method to compute $W^0, W^1, ..., W^{T-1}$ times UUU (and optionally the bias sequence) using divide-and-conquer / exponentiation-by-squaring style batching, rather than naive multiplying WWW T times. If the notebook already provides scaffolding (e.g., a function stub like `make_kernel_divide_conquer(W, U, T)`), fill it in. If not, implement a helper that computes the sequence faster than $O(T)$ matrix multiplications when possible. If dimension sizes make D&C not meaningfully faster, still implement D&C as requested. (c) Runtime comparison + explanation Time both methods across reasonable T (and maybe H) settings in the notebook. Explain in plain terms: loop is $O(T)$ sequential and not parallel across time conv can use optimized kernels, better cache/parallelism BUT kernel construction cost may dominate for small T `Conv1d` overhead / `im2col` vs direct conv may matter.

GPU notebook (d) Port both implementations to GPU Move tensors + modules to CUDA correctly (device = 'cuda' if ...). Make sure timing uses: `torch.cuda.synchronize()` before/after timed region warmup iterations (e) Compare runtime + explain Same fairness rules as CPU; discuss GPU parallelism and why conv tends to win for larger T . (f) Diagonal W + depthwise convolution optimization Constraint: WWW is diagonal. Then $W^\ell W^{\ell'} = W^{\ell+\ell'}$ is just elementwise powers of the diagonal: if w_{diag} is [H], $(W^\ell)_{diag} = w_{diag}^\ell$ Kernel becomes per-channel (depthwise) because each hidden dimension evolves independently. Implement a depthwise `Conv1d`: set groups = H input channels = H output channels = H You must map input contribution properly: If UUU is still a full matrix, you can first compute $z_t = U x_{t,t} = U x_t$ giving [H] per time, then apply depthwise conv with kernel $w^\ell w^{\ell'}$. Bias contribution: compute $\sum_{\ell=0}^{t-1} W^\ell b$ per channel efficiently (closed form if $|w|!=1$, or cumulative if needed) and add it. Ensure correctness vs the recurrent baseline (use `torch.testing.assert_close` with tolerances).

Output expectations For each TODO you fill: Provide the exact code to paste into the notebook cell. Add minimal comments explaining shapes and causal alignment. Add a correctness test comparing conv vs recurrent for small random sizes. Add timing code that is fair (warmup + synchronize on GPU). If any notebook expectation is unclear, infer it from surrounding code and match it (e.g., return shapes, dtype, batch dimension). Don't invent new APIs. Start by scanning both notebooks, listing all TODO locations, then implement them in order.

Planner Response

I'll start by scanning both notebooks to identify all TODO locations, then implement them systematically.

Read Jupyter notebook

Read Jupyter notebook

This is the typical prompt to give to an LLM to have it perform better and more accurately on questions

Planner Response

Now I have a clear picture of all the TODOs. Let me create a plan and implement them:

Updated todo list

Planner Response

Now I'll implement all the TODOs. Let me start with the CPU notebook:

Edited Jupyter notebook

Edited Jupyter notebook

Edited Jupyter notebook

Updated todo list

I suppose this can be seen as the literal hard prompt version of 'soft prompting'

Planner Response

Now let me implement the GPU notebook. The code is the same since PyTorch handles device placement automatically (the GPU notebook sets `torch.set_default_device('cuda')`):

Edited Jupyter notebook

Edited Jupyter notebook

Edited Jupyter notebook

This entire implementation took around 2 minutes to complete, which is very fast, and we will see that it completes all questions correctly without error

Planner Response

Now let me implement the diagonal W optimizations in the GPU notebook:

Edited Jupyter notebook

Edited Jupyter notebook

Edited Jupyter notebook

Updated todo list

Planner Response

All TODOs have been implemented. Let me verify the implementations by reading the notebooks to confirm the code is correct:

Read Jupyter notebook

Planner Response

The CPU notebook looks correct. Let me also verify the GPU notebook:

Read Jupyter notebook

Planner Response

All implementations are complete. Let me provide a summary of what was implemented:

Summary

Implementations Completed

CPU Notebook (`q_coding_ssm_forward_cpu.ipynb`)

1. `unrolled_ssm_forward` (cell 5): Recurrent loop implementation
 - Iterates over T time steps
 - Uses efficient batched matrix operations: `h @ W.T + x[:, t] @ U.T + b`
 - Output shape: (N, T, H)
2. `make_conv_kernel` (cell 7): Divide-and-conquer kernel construction
 - Builds $W^0, W^1, \dots, W^{T-1}$ using doubling strategy
 - Each iteration doubles the number of computed powers via batched matmul
 - Complexity: $O(\log T)$ matrix multiplications instead of $O(T)$
 - Output shape: (H, H, T) where $K[:, :, 1] = W^1$
3. `conv_ssm_forward` (cell 8): Convolution-based forward pass
 - Computes `s = x @ U.T + b` then permutes to (N, H, T)
 - Flips kernel for causal alignment
 - Applies left padding (T-1, 0) for causality
 - Uses `F.conv1d` with kernel shape (H, H, T)

This summary is helpful in understanding what changes the model made, and we will analyze the model's code output on the coding questions

GPU Notebook (`q_coding_ssm_forward_gpu.ipynb`)

1. Same implementations for `unrolled_ssm_forward`, `make_conv_kernel`, `conv_ssm_forward` (cells 5, 7, 8)
 - Code is device-agnostic; tensors inherit device from inputs
2. `make_diag_depthwise_kernel` (cell 15): Diagonal W kernel
 - Extracts diagonal `w_diag = torch.diag(W)`
 - Computes `w_diag[i]^l` for all `l = 0..T-1` using vectorized exponentiation
 - Output shape: (H, 1, T) for depthwise conv
3. `diag_conv_ssm_forward` (cell 16): Depthwise convolution
 - Uses `groups=H` for depthwise separable convolution
 - Each hidden dimension evolves independently
 - Much more efficient than full convolution
4. `diag_unrolled_ssm_forward` (cell 18): Optimized recurrent for diagonal W
 - Replaces `h @ W.T` with elementwise `w_diag * h`
 - Reduces per-step complexity from $O(H^2)$ to $O(H)$

Here we can see the LLM made key design decisions that allowed it to write efficient code. The divide and conquer question was especially efficient as it requires breaking down the exponent into powers of 2

Key Design Decisions

- **Causal alignment:** Kernel is flipped (`torch.flip(kernel, dims=[2])`) and left-padded (`F.pad(s, (T-1, 0))`) to ensure `h[t]` only depends on `s[0:t]`

- **Divide-and-conquer:** Doubles computed powers each iteration—if we have $W^0 \dots W^{k-1}$, we compute $W^k \dots W^{2k-1}$ via $W^k @ [W^0 \dots W^{k-1}]$
- **Device handling:** All tensors created with `device=x.device` and `dtype=x.dtype` for GPU compatibility

User Input

For the cpu ipynb: sanity check output: Unrolled h(t): tensor([[[[-1.2720e-02, 1.7478e-01, 1.1702e-01, -1.8475e-01], [4.3287e-02, 1.7190e-01, 7.0203e-02, 2.0290e-01], [5.2788e-03, 1.1071e-01, 4.4593e-02, 2.2496e-01], [-3.0554e-01, 3.6337e-01, -1.6552e-01, -5.3192e-01], [1.0813e-01, 5.4074e-01, 6.1866e-02, -1.0820e-01], [-2.6562e-02, 3.5316e-01, 2.7978e-02, 4.6655e-02], [-4.7121e-02, -9.5886e-02, 3.1475e-02, 2.7231e-01], [-1.4480e-01, 2.0062e-01, -1.8574e-01, 1.4919e-01]],

```
[[ 2.0503e-01,  1.1027e-01,  2.8894e-01,  1.3814e-01],
 [ 4.6919e-04,  3.3735e-01,  7.8319e-02,  3.0717e-02],
 [-2.1996e-02,  7.8201e-02,  8.5306e-02,  6.4196e-02],
 [ 9.9243e-03,  2.1734e-01,  4.4236e-03,  1.9512e-01],
 [-1.0414e-01,  2.8412e-01, -8.0947e-02,  1.0402e-02],
 [-1.3561e-01,  4.3784e-01, -1.1956e-01, -2.1006e-01],
 [ 2.1366e-01,  6.7647e-02,  2.8377e-01,  2.1382e-01],
 [ 1.3880e-02,  1.7214e-01,  1.7150e-01, -3.1018e-02]]])
```

Conv-based h(t): tensor([[[[-1.2720e-02, 1.7478e-01, 1.1702e-01, -1.8475e-01], [4.3287e-02, 1.7190e-01, 7.0203e-02, 2.0290e-01], [5.2788e-03, 1.1071e-01, 4.4593e-02, 2.2496e-01], [-3.0554e-01, 3.6337e-01, -1.6552e-01, -5.3192e-01], [1.0813e-01, 5.4074e-01, 6.1866e-02, -1.0820e-01], [-2.6562e-02, 3.5316e-01, 2.7978e-02, 4.6655e-02], [-4.7121e-02, -9.5886e-02, 3.1475e-02, 2.7231e-01], [-1.4480e-01, 2.0062e-01, -1.8574e-01, 1.4919e-01]],

```
[[ 2.0503e-01,  1.1027e-01,  2.8894e-01,  1.3814e-01],
 [ 4.6919e-04,  3.3735e-01,  7.8319e-02,  3.0717e-02],
 [-2.1996e-02,  7.8201e-02,  8.5306e-02,  6.4196e-02],
 [ 9.9243e-03,  2.1734e-01,  4.4236e-03,  1.9512e-01],
 [-1.0414e-01,  2.8412e-01, -8.0947e-02,  1.0402e-02],
 [-1.3561e-01,  4.3784e-01, -1.1956e-01, -2.1006e-01],
 [ 2.1366e-01,  6.7647e-02,  2.8377e-01,  2.1382e-01],
 [ 1.3880e-02,  1.7214e-01,  1.7150e-01, -3.1018e-02]]])
```

Here I provided the resulting output code so it could answer the written questions

Max absolute difference: 2.9802322387695312e-08

answer the written questions 1-4

Planner Response

Based on the sanity check output showing the implementations are correct (max absolute difference $\sim 3e-8$, which is floating-point precision), here are the answers to Questions 1-4:

Question 1

This makes sense for the answer, as a correct implementation would have very small absolute difference

What maximum absolute difference do you observe between the outputs of the two implementations?

The maximum absolute difference is 2.98×10^{-8} (approximately $3e-8$). This is within expected floating-point precision for float32 operations, confirming both implementations are numerically equivalent.

Question 2

What is the number of operations being performed in the recurrence-based implementation of the SSM forward pass?

For each time step t , we compute:

- $h @ W.T$: matrix-vector multiply $\rightarrow O(N \times H^2)$ operations
- $x[:, t] @ U.T$: matrix-vector multiply $\rightarrow O(N \times H \times D)$ operations
- Addition of bias and previous terms $\rightarrow O(N \times H)$ operations

These latex responses aren't rendered here, but in the chat they were. Helpful for clarity.

Over T time steps, total complexity is: $O(T \cdot N \cdot (H^2 + H \cdot D))$

Regarding these time complexity operations, they are correct as well

Or simplified: $O(T \cdot N \cdot H^2)$ when $H \geq D$ (which is common).

Key characteristic: The T iterations are **sequential** and cannot be parallelized since h_{t+1} depends on h_t .

Question 3

What is the number of operations being performed in the convolution-based implementation of the SSM forward pass?

1. **Kernel construction** (`make_conv_kernel`):
 - Divide-and-conquer computes $W^0, W^1, \dots, W^{T-1}$
 - $O(\log T)$ batched matrix multiplications, each $O(T \times H^3) \rightarrow O(T \cdot H^3 \cdot \log T)$
2. **Input projection** $s = x @ U.T + b : O(N \times T \times H \times D)$
3. **Convolution** `F.conv1d`:
 - Input: (N, H, T) , Kernel: (H, H, T)
 - Standard conv1d complexity: $O(N \times H^2 \times T^2)$
 - (Or $O(N \times H^2 \times T \log T)$ if using FFT-based convolution)

Total complexity: $O(N \cdot H^2 \cdot T^2 + T \cdot H^3 \cdot \log T)$

Question 4

This table that Windsurf LLM produced displays clear and accurate characteristics of each method

Compare the trade-off between the two implementations.

Aspect	Recurrent	Convolution
Time complexity	$O(T \cdot N \cdot H^2)$	$O(N \cdot H^2 \cdot T^2 + T \cdot H^3 \cdot \log T)$
Parallelism	Sequential over T (cannot parallelize time)	Fully parallelizable (conv is one batched op)
Memory	$O(N \cdot H)$ working memory	$O(H^2 \cdot T)$ for kernel + $O(N \cdot H \cdot T)$ for padded input
Kernel overhead	None	Must build $W^0..W^{T-1}$ before conv

Trade-offs:

1. **Small T**: Recurrent wins because kernel construction overhead dominates, and the sequential loop has low constant factors.
2. **Large T**: Convolution wins on GPU because:
 - Conv1d leverages highly optimized CUDA kernels
 - Parallelism across time eliminates T sequential kernel launches
 - Better cache utilization and memory bandwidth
3. **Small H, large T**: Convolution benefits most (kernel construction is cheap, parallelism over T is valuable).
4. **Large H**: Recurrent may be competitive since both scale as H^2 , but kernel construction adds H^3 factor.

Bottom line: Convolution trades increased FLOPs for parallelism. On modern hardware (especially GPUs), this trade-off favors convolution for longer sequences.

User Input

answer question 5 based on these graphs

Planner Response

Question 5

What do you observe about the runtime of the two implementations as T and H increase? Explain your finding.

Observations

Effect of increasing T:

- Both implementations show increasing runtime as T grows (expected)
- The unrolled method scales roughly linearly with T ($O(T)$)
- The convolution method scales super-linearly with T ($O(T^2)$) due to conv1d with kernel size T

Effect of increasing H:

- At **H=2**: Conv is faster than unrolled for large T ($T \geq 32$)
- At **H=4**: Methods are roughly comparable, with conv slightly faster at large T
- At **H=8**: Crossover point—conv starts becoming slower than unrolled at large T
- At **H=16, H=32**: Unrolled is consistently faster, and the gap widens significantly with T

Key pattern: As H increases, the convolution method becomes increasingly slower relative to unrolled.

Explanation

1. **Unrolled complexity:** $O(T \cdot H^2)$ — linear in T, quadratic in H

Windsurf goes into a clear analysis of the graph at each value of H.

- Each of T steps does an $H \times H$ matrix multiply

2. Convolution complexity: $O(T \cdot H^3 \cdot \log T)$ for kernel + $O(H^2 \cdot T^2)$ for conv1d

- Kernel construction requires computing $W^0 \dots W^{T-1}$, each $H \times H$
- Conv1d with (H, H, T) kernel on (N, H, T) input scales as $H^2 \cdot T^2$

3. Why conv loses at large H:

- The H^3 factor in kernel construction dominates
- The $H^2 \cdot T^2$ conv operation grows faster than $H^2 \cdot T$ unrolled
- On CPU, conv1d cannot leverage parallelism effectively—the sequential overhead of the unrolled loop is less costly than the raw FLOP count of convolution

4. Why conv wins at small H:

- Kernel construction overhead is minimal (H^3 is small)
- Conv1d has better cache locality and vectorization for small H
- Python loop overhead in unrolled becomes relatively more significant

Summary

On CPU, the **unrolled recurrent method is generally faster**, especially as H increases. The convolution approach has higher asymptotic complexity ($O(T^2)$ vs $O(T)$) and significant kernel construction overhead ($O(H^3)$). The convolution method only wins for very small H where its vectorized operations outweigh the loop overhead of the recurrent approach. This trend would likely reverse on GPU where parallelism can be exploited.

User Input

Yes. Here Windsurf notices that right now. the CPU performs better, but the GPU will likely cause a trend reversal

Output of first sanity check: Unrolled h(t): tensor([[[[0.2619, -0.3655, 0.0208, 0.2176], [0.3612, -0.1343, 0.1213, 0.1395], [0.1538, -0.0023, -0.1271, -0.0176], [0.3927, -0.1537, 0.1103, 0.0732], [0.0711, -0.0081, -0.2333, -0.0349], [0.3832, 0.1360, -0.0033, -0.1085], [0.3630, -0.0947, 0.0646, 0.0288], [0.1400, 0.0112, -0.1930, -0.0218]], device='cuda:0']])

```
[[[-0.0384,  0.0034, -0.3276, -0.1926],
 [ 0.3462,  0.0823, -0.0832, -0.0996],
 [ 0.1896,  0.2200, -0.2117, -0.2180],
 [ 0.0559,  0.0817, -0.3868, -0.1649],
 [ 0.4200, -0.4191,  0.1368,  0.2055],
 [ 0.2792, -0.0009,  0.0220,  0.0395],
 [ 0.1535, -0.3252, -0.0715,  0.1442],
 [ 0.2363, -0.3806,  0.0232,  0.2383]]], device='cuda:0')
```

Conv-based h(t): tensor([[[[0.2619, -0.3655, 0.0208, 0.2176], [0.3612, -0.1343, 0.1213, 0.1395], [0.1538, -0.0023, -0.1271, -0.0176], [0.3927, -0.1537, 0.1103, 0.0732], [0.0711, -0.0081, -0.2333, -0.0349], [0.3832, 0.1360, -0.0033, -0.1085], [0.3630, -0.0947, 0.0646, 0.0288], [0.1400, 0.0112, -0.1930, -0.0218]], device='cuda:0']])

```
[[[-0.0384,  0.0034, -0.3276, -0.1926],
 [ 0.3462,  0.0823, -0.0832, -0.0996],
 [ 0.1896,  0.2200, -0.2117, -0.2180],
 [ 0.0559,  0.0817, -0.3868, -0.1649],
 [ 0.4200, -0.4191,  0.1368,  0.2055],
 [ 0.2792, -0.0009,  0.0220,  0.0395],
 [ 0.1535, -0.3252, -0.0715,  0.1442],
 [ 0.2363, -0.3806,  0.0232,  0.2383]]], device='cuda:0')
```

Max absolute difference: 5.960464477539063e-08

[I have attached 8 images for the output of implementation complexity]

Output of the second sanity check: Unrolled h(t): tensor([[[[-0.0563, 0.0190, 0.0105, -0.1580], [0.1031, -0.0293, 0.1256, -0.2023], [0.0925, -0.0149, 0.0961, -0.4178], [-0.0540, -0.0726, 0.0494, -0.2727], [-0.4944, -0.1865, -0.0981, 0.3956], [0.0738, -0.1133, 0.1744, 0.1411], [-0.1444, -0.1421, 0.0544, 0.0483], [-0.2369, -0.1165, 0.0221, 0.4350]], device='cuda:0']])

```
[[[-0.3021, -0.0758, -0.0509,  0.3044],
 [ 0.1761, -0.0469,  0.1658, -0.3138],
 [-0.3661, -0.1767, -0.0704, -0.1081],
 [-0.5248, -0.1941, -0.1096,  0.4271],
 [ 0.1389, -0.0366,  0.1629, -0.0441],
 [-0.2679, -0.1284, -0.0054,  0.2542],
 [-0.5545, -0.1520, -0.1616,  0.2249],
 [-0.0282, -0.0786,  0.0734, -0.1926]]], device='cuda:0')
```

Conv-based h(t): tensor([[[[-0.0563, 0.0190, 0.0105, -0.1580], [0.1031, -0.0293, 0.1256, -0.2023], [0.0925, -0.0149, 0.0961, -0.4178], [-0.0540, -0.0726, 0.0494, -0.2727], [-0.4944, -0.1865, -0.0981, 0.3956], [0.0738, -0.1133, 0.1744, 0.1411], [-0.1444, -0.1421, 0.0544, 0.0483], [-0.2369, -0.1165, 0.0221, 0.4350]], device='cuda:0']])

```
[[[-0.3021, -0.0758, -0.0509, 0.3044],
 [ 0.1761, -0.0469, 0.1658, -0.3138],
 [-0.3661, -0.1767, -0.0704, -0.1081],
 [-0.5248, -0.1941, -0.1096, 0.4271],
 [ 0.1389, -0.0366, 0.1629, -0.0441],
 [-0.2679, -0.1284, -0.0054, 0.2542],
 [-0.5545, -0.1520, -0.1616, 0.2249],
 [-0.0282, -0.0786, 0.0734, -0.1926]]], device='cuda:0')
```

Max absolute difference: 2.9802322387695312e-08

[I have attached 8 more images of the Measure Runtime with Optimization]

answer the written questions for the GPU notebook

Planner Response

GPU Notebook Written Answers

Question 6

What do you observe about the runtime of the two implementations as T and H increase? Do you observe the same trend as the CPU notebook? Explain your reasoning.

Observations (Images 1-8: Unstructured W)

Effect of T:

- **Unrolled:** Runtime grows linearly with T (as expected from O(T) sequential steps)
- **Conv:** Runtime stays nearly flat for small/medium H, grows slowly with T

Effect of H:

- **H=4 to H=64:** Conv is dramatically faster (10-100x) than unrolled for all T values
- **H=128, H=256:** Conv shows a spike at T=256, but still competitive or faster overall
- **H=512:** Unusual behavior with spikes, likely due to memory/kernel launch overhead

Comparison with CPU

This is the OPPOSITE trend from CPU:

Aspect	CPU	GPU
Winner for small H	Conv (slightly)	Conv (dramatically)
Winner for large H	Unrolled	Conv (still wins)
Conv scaling with T	Poor (O(T²))	Nearly flat

A clear comparison of CPU vs GPU with a very helpful underlying explanation

Explanation

1. **GPU parallelism:** Conv1d on GPU executes all T output positions in parallel using thousands of CUDA cores. The unrolled loop cannot parallelize across time—each step requires T sequential kernel launches.
2. **Kernel launch overhead:** Each iteration of the unrolled loop launches a separate CUDA kernel. With T=512, that's 512 kernel launches, each with ~10-100µs overhead. This dominates the actual compute time.
3. **Conv1d optimization:** cuDNN's conv1d implementation is highly optimized with fused operations, better memory coalescing, and reduced kernel launch overhead (single kernel for entire operation).
4. **Why conv wins on GPU but not CPU:**
 - CPU cannot parallelize conv1d across T effectively (limited cores)
 - GPU has massive parallelism that conv1d exploits
 - The sequential loop overhead on CPU is just Python iteration; on GPU it's expensive kernel launches

Question 7

What do you observe here? How do your findings differ from the unstructured matrix case? Explain your reasoning.

Observations (Images 9-16: Diagonal W with Depthwise Conv)

Key finding: Depthwise convolution is **overwhelmingly faster** than unrolled—by 50-100x across all H and T values.

- **Diag-Conv runtime:** Nearly flat (~0.001s) regardless of T or H
- **Unrolled runtime:** Grows linearly with T, reaches ~0.05s at T=512
- The gap is consistent across all H values (4 to 512)

The Windsurf LLM even analyzes the time values in the graph images, not something I would have normally expected.

Comparison with Unstructured Case

Aspect	Unstructured (Full Conv)	Diagonal (Depthwise Conv)
Conv advantage	10-100x faster	50-100x faster
Conv scaling with H	Degrades at $H \geq 128$	Stays flat
Spikes/instability	Yes ($H=128, 256, 512$)	No—very stable

Explanation

1. **Depthwise conv is much cheaper:**
 - Full conv: kernel shape $(H, H, T) \rightarrow O(H^2 \cdot T)$ parameters, $O(H^2 \cdot T^2)$ compute
 - Depthwise conv: kernel shape $(H, 1, T) \rightarrow O(H \cdot T)$ parameters, $O(H \cdot T)$ compute
 - Removing the H^2 factor eliminates the bottleneck seen at large H
2. **Perfect GPU utilization:** Depthwise conv with `groups=H` processes each channel independently. This maps perfectly to GPU parallelism—each of the H channels can be processed by different thread blocks with no cross-channel dependencies.
3. **Kernel construction is trivial:** Computing `w_diag^1` is just elementwise exponentiation $O(H \cdot T)$, versus $O(H^3 \cdot \log T)$ for matrix powers in the unstructured case.
4. **Why unrolled doesn't benefit as much from diagonal W :**
 - Even though each step is $O(H)$ instead of $O(H^2)$, we still have T sequential kernel launches
 - The kernel launch overhead dominates, not the per-step compute
 - Diagonal optimization helps compute but not the sequential bottleneck

Summary

The diagonal constraint enables depthwise convolution, which is dramatically more efficient than full convolution. This eliminates both the H^2 compute factor and the memory bandwidth bottleneck, making the conv approach consistently superior on GPU regardless of problem size. This is why modern SSMS (like Mamba) use diagonal or structured state matrices—it enables efficient parallel computation.



Introduction

In this notebook, we'll implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.

You will use CPU for this notebook.

Imports

```
In [ ]: import time
import math
import matplotlib.pyplot as plt

import torch
import numpy as np
import torch.nn.functional as F
```

SSM Update Rule

We consider an Linear RNN described by the update:

$$h_{t+1} = W h_t + U x_t + b$$

for $t = 0, 1, \dots, T - 1$. The variables are:

- $h_t \in \mathbb{R}^H$, the hidden state at time t .
- $x_t \in \mathbb{R}^{N \times D}$, the input at time t .
- $W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix.
- $U \in \mathbb{R}^{H \times D}$, the input projection matrix.
- $b \in \mathbb{R}^H$, the bias vector.

N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H .

Below you will implement the forward pass for the SSM using recursion based approach. The `unrolled_ssm_forward` function will take weights W , U , b and input x and return the hidden states h across different time steps.

```
In [ ]: def unrolled_ssm_forward(W, U, b, x):
        """
        Unroll the linear RNN in time:
         $h_{t+1} = W h_t + U x_t + b$ 
        """
```



```
with initial  $h_0 = 0$ .
```

Args:

W: (H, H) weight matrix

U: (H, D) input projection

b: (H,) bias

x: (N, T, D) input sequence over T steps

Returns:

h_all: (N, T, H) hidden states for $t=1..T$

(h_all[t] corresponds to h_{t+1} in the usual notation).

```
"""
```

```
#####  
#                               TODO: Implement the recurrent pass here
```

```
#####
```

```
N, T, D = x.shape
```

```
H = W.shape[0]
```

```
# Preallocate output tensor: (N, T, H)
```

```
h_all = torch.zeros(N, T, H, dtype=x.dtype, device=x.device)
```

```
# Initialize hidden state  $h_0 = 0$ , shape (N, H)
```

```
h = torch.zeros(N, H, dtype=x.dtype, device=x.device)
```

```
for t in range(T):
```

```
    #  $h_{t+1} = W @ h_t + U @ x_t + b$ 
```

```
    #  $x[:, t, :]$  has shape (N, D)
```

```
    #  $h @ W.T$  gives (N, H),  $x[:, t] @ U.T$  gives (N, H)
```

```
    h = h @ W.T + x[:, t] @ U.T + b # (N, H)
```

```
    h_all[:, t, :] = h
```

```
#####
```

```
#                               END OF YOUR CODE
```

```
#####
```

```
return h_all
```

Surprisingly, this is the EXACT staff solution (with extra comments). These extra comments help to understand what is going on in this code

Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

- `make_conv_kernel(W, T)` : This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K . Given that T is a power of 2, you can implement this using a divide and conquer based approach.
- `conv_ssm_forward(W, U, b, x)` : This function will take weights W ,

U , b and input x and return the hidden states h across different time steps.

```
In [ ]: def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using convld.

    Args:
        W: (H, H) weight matrix
        T: scalar

    Returns:
        kernel_for_conv: (H, H, T) tensor
    """
    #####
    #                                TODO: Implement the kernel here
    #####
    # We need  $K[:, :, l] = W^l$  for  $l = 0, 1, \dots, T-1$ 
    # Using divide-and-conquer: compute powers of  $W$  efficiently
    #
    # Approach: Build  $W^0, W^1, \dots, W^{T-1}$  using repeated squaring pattern
    # Start with  $W^0 = I$ , then double the sequence length each iteration

    H = W.shape[0]
    device = W.device
    dtype = W.dtype

    # powers[i] will hold  $W^i$ , stored as list then stacked
    # Start with  $W^0 = I$ 
    powers = [torch.eye(H, dtype=dtype, device=device)] # [ $W^0$ ]

    # Divide-and-conquer: if we have  $W^0..W^{k-1}$ , we can compute  $W^k..W^{2k-1}$ 
    # by multiplying each by  $W^k$ 
    current_len = 1
    while current_len < T:
        #  $W^{\{current\_len\}} = W^{\{current\_len\}}$  (the next power we need)
        # Stack current powers: shape (current_len, H, H)
        current_stack = torch.stack(powers, dim=0) # (current_len, H, H)

        #  $W^{\{current\_len\}}$  is powers[-1] @ W
        W_power = powers[-1] @ W # This is  $W^{\{current\_len\}}$ 

        # Compute  $W^{\{current\_len + i\}} = W^{\{current\_len\}} @ W^i$  for  $i = 0..current\_len-1$ 
        # new_powers[i] = W_power @  $W^i = W^{\{current\_len\}} @ W^i = W^{\{current\_len + i\}}$ 
        # But we want  $W^{\{current\_len\}}, W^{\{current\_len+1\}}, \dots, W^{\{2*current\_len-1\}}$ 
        # new_stack[i] =  $W^{\{current\_len\}} @ powers[i] = W^{\{current\_len + i\}}$ 
        new_stack = W_power @ current_stack # (current_len, H, H)

        # Add new powers to list
        for i in range(current_len):
            if len(powers) >= T:
                break
            powers.append(new_stack[i])

    # Stack all powers into a single tensor
    kernel_for_conv = torch.stack(powers, dim=0) # (T, H, H)
    return kernel_for_conv
```

Here, Windsurf uses a slightly different implementation, yet still arrives at the right answer. Windsurf LLM uses a loop to torch.stack powers while the staff solution uses a list to store powers and torch.cat.

```

        break
    powers.append(new_stack[i])

    current_len = len(powers)

# Stack all powers: (T, H, H)
powers_tensor = torch.stack(powers[:T], dim=0) # (T, H, H)

# Kernel shape for conv1d: (H, H, T)
#  $K[:, :, l] = W^l$  means kernel_for_conv = powers_tensor.permute(1, 2, 0)
kernel_for_conv = powers_tensor.permute(1, 2, 0) # (H, H, T)
#####
#                                     END OF YOUR CODE
#####
return kernel_for_conv

```

```

In [ ]: def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)
    #####
    #                                     TODO: Implement the convolution here
    #####
    # s has shape (N, H, T) - this is our "input" to conv1d
    # kernel has shape (H, H, T) where  $K[:, :, l] = W^l$ 
    #
    # The recurrence expands to:
    #  $h_t = \sum_{k=0}^{t-1} W^{t-1-k} * s_{k+1}$ 
    # where  $s_k = U @ x_{k-1} + b$  (with 1-indexing in math, 0-indexing in code)
    # In code (0-indexed):  $h[t] = \sum_{k=0}^{t-1} W^{t-1-k} * s[k]$ 
    #  $= \sum_{l=0}^{t-1} W^l * s[t-1-l]$  (substituting  $l = t-1-k$ )
    #

```

Windsurf LLM produces detailed comments before making the TODOs, this can be interpreted as either providing comments for the user to interpret or writing out comments that help it 'think' in the right way.

Windsurf's LLM's solution here again is slightly different from the staff solution but still achieves the same answer. Both solutions pad by T-1 to allow for correct conv implementation, but the Windsurf LLM flips the kernel first. However in the staff solution, they implemented 'make_conv_kernel_fast', which already has the tensor in the correct dimensions.

As such the Windsurf LLM needs to flip ours since we use the original make_conv_kernel

```
# This is a causal convolution. Conv1d computes:
#   out[t] = sum_{l=0}^{K-1} kernel[l] * input[t + l - padding]
#
# For causal conv with kernel size T, we need padding = T-1 on the left
# and we flip the kernel in time so that kernel[l] corresponds to W^l
#
# Conv1d expects:
#   input: (N, C_in, L) = (N, H, T)
#   weight: (C_out, C_in, K) = (H, H, T)
#
# We need to flip kernel along time dimension for proper causal alignment
kernel_flipped = torch.flip(kernel, dims=[2]) # (H, H, T)

# Apply causal padding (T-1 on left, 0 on right)
s_padded = F.pad(s, (T - 1, 0)) # (N, H, 2T-1)

# Perform convolution
# F.conv1d: input (N, C_in, L), weight (C_out, C_in, K)
h_conv = F.conv1d(s_padded, kernel_flipped) # (N, H, T)

# Permute back to (N, T, H)
h_all = h_conv.permute(0, 2, 1) # (N, T, H)
#####
#                                     END OF YOUR CODE
#####
return h_all
```

Sanity Check

We can compare the outputs of the two implementations to check if they are consistent.

```
In [ ]: def sanity_check():
    T = 8 # number of time steps
    H = 4 # hidden dimension
    D = 3 # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
```

```

print("Unrolled h(t):")
print(h_unrolled)
print("\nConv-based h(t):")
print(h_conv)
print("\nMax absolute difference:", diff.item())

sanity_check()

```

This answer is correct as we get 10^{-8} error in our difference

Question 1

What maximum absolute difference do you observe between the outputs of the two implementations for the following inputs?

Answer: The maximum absolute difference is approximately 2.98×10^{-8} (around $3e-8$). This is within expected floating-point precision for float32 operations, confirming both implementations are numerically equivalent.

Implementation Complexity

We will now compare the two implementation in terms of their runtime efficiencies. But before we proceed, answer the following question.

Question 2

What is the number of operations being performed in the recurrence based implementation of the SSM forward pass?

Answer: For each time step t , we compute:

- $h @ W.T$: matrix-vector multiply $\rightarrow O(N \times H^2)$ operations
- $x[:, t] @ U.T$: matrix-vector multiply $\rightarrow O(N \times H \times D)$ operations
- Addition of bias and previous terms $\rightarrow O(N \times H)$ operations

Over T time steps, total complexity is: $O(T \cdot N \cdot (H^2 + H \cdot D))$, or simplified $O(T \cdot N \cdot H^2)$ when $H \geq D$.

Key characteristic: The T iterations are **sequential** and cannot be parallelized since h_{t+1} depends on h_t .

Question 3

What is the number of operations being performed in the convolution based implementation of the SSM forward pass?

Mathematically, Windsurf breaks down the time complexity, leading to a correct runtime calculation

Answer:

Again, Windsurf mathematically derives the time complexity, and this is impressive as there are a lot of variables to worry about throughout the notebook.

1. **Kernel construction** (`make_conv_kernel`):
 - Divide-and-conquer computes $W^0, W^1, \dots, W^{T-1}$
 - $O(\log T)$ batched matrix multiplications, each $O(T \times H^3) \rightarrow$
 $O(T \cdot H^3 \cdot \log T)$
2. **Input projection** `s = x @ U.T + b`: **$O(N \times T \times H \times D)$**
3. **Convolution** `F.conv1d`:
 - Input: (N, H, T) , Kernel: (H, H, T)
 - Standard conv1d complexity: **$O(N \times H^2 \times T^2)$**

Total complexity: **$O(N \cdot H^2 \cdot T^2 + T \cdot H^3 \cdot \log T)$**

Question 4

Compare the trade-off if any between the two implementations based on your answers above.

Answer:

Aspect	Recurrent	Convolution
Time complexity	$O(T \cdot N \cdot H^2)$	$O(N \cdot H^2 \cdot T^2 + T \cdot H^3 \cdot \log T)$
Parallelism	Sequential over T	Fully parallelizable
Memory	$O(N \cdot H)$ working memory	$O(H^2 \cdot T)$ for kernel + $O(N \cdot H \cdot T)$ for padded input
Kernel overhead	None	Must build $W^0..W^{T-1}$ before conv

Trade-offs:

Windsurf effectively outputs a table with the trade-offs of each implementation

- **Small T** : Recurrent wins because kernel construction overhead dominates
- **Large T** : Convolution can win on parallel hardware (GPU) due to parallelism across time
- **Small H , large T** : Convolution benefits most (kernel construction is cheap)
- **Large H** : Recurrent may be competitive since kernel construction adds H^3 factor

Convolution trades increased FLOPs for parallelism—beneficial on GPUs, less so on

CPUs.

Runtime Comparison

```
In [ ]: import ipywidgets as widgets
        from ipywidgets import interact

def measure_runtime(method_fn, W, U, b, x, warmup=1, repeats=10):
    # Warm-up runs (ignored in timing):
    for _ in range(warmup):
        method_fn(W, U, b, x)

    # Timed runs:
    start = time.time()
    for _ in range(repeats):
        method_fn(W, U, b, x)
    end = time.time()

    avg_time = (end - start) / repeats
    return avg_time

def run():
    T_values_cache = {}
    times_unrolled_vs_T_cache = {}
    times_conv_vs_T_cache = {}

    for H in [2, 4, 8, 16, 32]:
        # We'll keep D, N fixed
        D = 32
        N = 32

        T_values = [8, 32, 128, 256, 512]

        # Build random U, b
        U = torch.randn(H, D)*0.1
        b = torch.randn(H)*0.1

        times_unrolled_vs_T = []
        times_conv_vs_T = []

        for T in T_values:

            diag_vals = torch.randn(H)*0.05
            W = torch.randn(H, H)*0.05
            x = torch.randn(N, T, D)

            t_unrolled = measure_runtime(unrolled_ssm_forward, W, U, b, x)

            t_conv = measure_runtime(conv_ssm_forward, W, U, b, x)
```

```

        times_unrolled_vs_T.append(t_unrolled)
        times_conv_vs_T.append(t_conv)

    T_values_cache[H] = T_values
    times_unrolled_vs_T_cache[H] = times_unrolled_vs_T
    times_conv_vs_T_cache[H] = times_conv_vs_T
    return T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache

T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache = run()

@interact(H=widgets.FloatLogSlider(min=1, max=5, base=2, value=4, step=1))
def interactive_benchmark(H):
    """
    Compare unrolled vs. diagonal-convolution RNN forward for various T,
    at a chosen hidden dimension H from the slider.
    """
    H = int(H)
    T_values = T_values_cache[H]
    T_unrolled = times_unrolled_vs_T_cache[H]
    T_conv = times_conv_vs_T_cache[H]

    # Plot
    plt.figure(figsize=(6,4))
    plt.plot(T_values, T_unrolled, label="Unrolled", marker='o')
    plt.plot(T_values, T_conv, label="Conv", marker='s')
    plt.title(f"Runtime vs T, H={H}")
    plt.xlabel("Time Steps (T)")
    plt.ylabel("Runtime (sec)")
    plt.yscale('log')
    plt.grid(True)
    plt.legend()
    plt.show()

```

Question 5

What do you observe about the runtime of the two implementations as T and H increase? Explain your finding.

Answer:

Observations:

- For this question, I fed the LLM sequential pictures of the graph at different values of H . Windsurf is able to parse through these images and identify the overall trend and reasoning behind why we arrive at these graphs
- **Effect of T :** Both methods slow down as T increases. Unrolled scales $\sim$ linearly, conv scales super-linearly.
 - **Effect of H :**
 - At $H=2$: Conv is faster for large T
 - At $H=4$: Methods are roughly comparable
 - At $H=8+$: Unrolled becomes faster, gap widens with H

Explanation:

1. **Unrolled:** $O(T \cdot H^2)$ — linear in T , quadratic in H
2. **Conv:** $O(T \cdot H^3 \cdot \log T)$ for kernel + $O(H^2 \cdot T^2)$ for conv1d

Windsurf even predicts that GPU can parallelize the convolution approach (which is what we will see in the next notebook)

Why conv loses at large H on CPU:

- The H^3 factor in kernel construction dominates
- Conv1d cannot leverage parallelism effectively on CPU
- Python loop overhead in unrolled is less costly than raw FLOP count of convolution

Why conv wins at small H :

- Kernel construction overhead is minimal
- Conv1d has better cache locality and vectorization

Bottom line: On CPU, the unrolled recurrent method is generally faster, especially as H increases. The convolution approach would benefit more from GPU parallelism.



Introduction

In this notebook, we'll implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.

You will need GPU for this notebook which can be enabled via changing the runtime.

You can copy your solutions from the `q_coding_ssm_forward_cpu` notebook for the first part of the notebook.

Imports

```
In [ ]: import time
import math
import matplotlib.pyplot as plt

import torch
import numpy as np
import torch.nn.functional as F

torch.set_default_device('cuda')
```

SSM Update Rule

We consider an RNN described by the update:

$$h_{t+1} = W h_t + U x_t + b$$

for $t = 0, 1, \dots, T - 1$. The variables are:

- $h_t \in \mathbb{R}^H$, the hidden state at time t .
- $x_t \in \mathbb{R}^{N \times D}$, the input at time t .
- $W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix.
- $U \in \mathbb{R}^{H \times D}$, the input projection matrix.
- $b \in \mathbb{R}^H$, the bias vector.

N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H .

Below you will implement the forward pass for the SSM using recursion based approach. The `unrolled_ssm_forward` function will take weights W , U , b

and input x and return the hidden states h across different time steps.

```
In [ ]: def unrolled_ssm_forward(W, U, b, x):
        """
        Unroll the linear RNN in time:
             $h_{t+1} = W h_t + U x_t + b$ 
        with initial  $h_0 = 0$ .

        Args:
            W: (H, H) weight matrix
            U: (H, D) input projection
            b: (H,) bias
            x: (N, T, D) input sequence over T steps
        Returns:
            h_all: (N, T, H) hidden states for t=1..T
                    (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).
        """
        #####
        #                                TODO: Implement the recurrent pass here
        #####
        N, T, D = x.shape
        H = W.shape[0]

        # Preallocate output tensor: (N, T, H)
        h_all = torch.zeros(N, T, H, dtype=x.dtype, device=x.device)

        # Initialize hidden state  $h_0 = 0$ , shape (N, H)
        h = torch.zeros(N, H, dtype=x.dtype, device=x.device)

        for t in range(T):
            #  $h_{t+1} = W @ h_t + U @ x_t + b$ 
            #  $x[:, t, :]$  has shape (N, D)
            #  $h @ W.T$  gives (N, H),  $x[:, t] @ U.T$  gives (N, H)
            h = h @ W.T + x[:, t] @ U.T + b # (N, H)
            h_all[:, t, :] = h

        #####
        #                                END OF YOUR CODE
        #####
        return h_all
```

Like in the previous notebook, the Windsurf LLM writes the exact staff solution (with extra comments for clarity)

Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

- `make_conv_kernel(W, T)` : This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K . Given that T is a power of 2, you can implement this using a divide and conquer based approach.
- `conv_ssm_forward(W, U, b, x)` : This function will take weights W , U , b and input x and return the hidden states h across different time steps.

```
In [ ]: def make_conv_kernel(W, T):
        """
        Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
        the ssm forward pass using convld.

        Args:
            W: (H, H) weight matrix
            T: scalar

        Returns:
            kernel_for_conv: (H, H, T) tensor
        """
        #####
        #                                TODO: Implement the kernel here
        #####
        # We need  $K[:, :, l] = W^l$  for  $l = 0, 1, \dots, T-1$ 
        # Using divide-and-conquer: compute powers of  $W$  efficiently
        #
        # Approach: Build  $W^0, W^1, \dots, W^{T-1}$  using repeated squaring pattern
        # Start with  $W^0 = I$ , then double the sequence length each iteration

        H = W.shape[0]
        device = W.device
        dtype = W.dtype

        # powers[i] will hold  $W^i$ , stored as list then stacked
        # Start with  $W^0 = I$ 
        powers = [torch.eye(H, dtype=dtype, device=device)] # [ $W^0$ ]

        # Divide-and-conquer: if we have  $W^0..W^{k-1}$ , we can compute  $W^k..W^{2k-1}$ 
        # by multiplying each by  $W^k$ 
        current_len = 1
        while current_len < T:
            #  $W^{\{current\_len\}} = W^{\{current\_len\}}$  (the next power we need)
            # Stack current powers: shape (current_len, H, H)
            current_stack = torch.stack(powers, dim=0) # (current_len, H, H)

            #  $W^{\{current\_len\}}$  is powers[-1] @ W
            W_power = powers[-1] @ W # This is  $W^{\{current\_len\}}$ 

            # Compute  $W^{\{current\_len + i\}} = W^{\{current\_len\}} @ W^i$  for  $i = 0..current\_len-1$ 
            # new_powers[i] =  $W\_power @ W^i = W^{\{current\_len\}} @ W^i = W^{\{current\_len + i\}}$ 
```

Windsurf's solution here is highly complicated (much more than the staff solution), yet it achieves the right answer. We build powers of 2 and then break down T by the base powers of 2. The staff solution uses a list comprehension, which might be a lot more expensive in compute

```

# But we want  $W^{\{current\_len\}}$ ,  $W^{\{current\_len+1\}}$ , ...,  $W^{\{2*current\_len\}}$ 
#  $new\_stack[i] = W^{\{current\_len\}} @ powers[i] = W^{\{current\_len + i\}}$ 
new_stack = W_power @ current_stack # ( $current\_len$ ,  $H$ ,  $H$ )

# Add new powers to list
for i in range(current_len):
    if len(powers) >= T:
        break
    powers.append(new_stack[i])

current_len = len(powers)

# Stack all powers: ( $T$ ,  $H$ ,  $H$ )
powers_tensor = torch.stack(powers[:T], dim=0) # ( $T$ ,  $H$ ,  $H$ )

# Kernel shape for conv1d: ( $H$ ,  $H$ ,  $T$ )
#  $K[:, :, l] = W^l$  means kernel_for_conv = powers_tensor.permute(1, 2, 0)
kernel_for_conv = powers_tensor.permute(1, 2, 0) # ( $H$ ,  $H$ ,  $T$ )
#####
#                                     END OF YOUR CODE
#####
return kernel_for_conv

```

```

In [ ]: def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: ( $H$ ,  $H$ ) weight matrix
        U: ( $H$ ,  $D$ ) input projection
        b: ( $H$ ,) bias
        x: ( $N$ ,  $T$ ,  $D$ ) input (batch= $N$ , time steps= $T$ , input dim= $D$ )

    Returns:
        h_all: ( $N$ ,  $T$ ,  $H$ ) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b

    s = s.permute(0, 2, 1)

    # Build the kernel with shape ( $H$ ,  $H$ ,  $T$ ).
    kernel = make_conv_kernel(W, T)
    #####
    #                                     TODO: Implement the convolution here
    #####
    # s has shape ( $N$ ,  $H$ ,  $T$ ) - this is our "input" to conv1d
    # kernel has shape ( $H$ ,  $H$ ,  $T$ ) where  $K[:, :, l] = W^l$ 
    #

```

Again in this code, Windsurf provides a detailed explanation in the comments when the solution is literally just 4 lines of code. While helpful, this might not be beneficial if one is trying to save tokens. Otherwise it provides clarity to the user of how the code was developed.

```
# The recurrence expands to:
#   h_t = sum_{k=0}^{t-1} W^{t-1-k} * s_{k+1}
# where s_k = U @ x_{k-1} + b (with 1-indexing in math, 0-indexing in code)
#
# In code (0-indexed): h[t] = sum_{k=0}^{t-1} W^{t-1-k} * s[k]
#   = sum_{l=0}^{t-1} W^l * s[t-1-l] (substituting l = t-1-k)
#
# This is a causal convolution. Conv1d computes:
#   out[t] = sum_{l=0}^{K-1} kernel[l] * input[t + l - padding]
#
# For causal conv with kernel size T, we need padding = T-1 on the left
# and we flip the kernel in time so that kernel[l] corresponds to W^l
#
# Conv1d expects:
#   input: (N, C_in, L) = (N, H, T)
#   weight: (C_out, C_in, K) = (H, H, T)
#
# We need to flip kernel along time dimension for proper causal alignment
kernel_flipped = torch.flip(kernel, dims=[2]) # (H, H, T)

# Apply causal padding (T-1 on left, 0 on right)
s_padded = F.pad(s, (T - 1, 0)) # (N, H, 2T-1)

# Perform convolution
# F.conv1d: input (N, C_in, L), weight (C_out, C_in, K)
h_conv = F.conv1d(s_padded, kernel_flipped) # (N, H, T)

# Permute back to (N, T, H)
h_all = h_conv.permute(0, 2, 1) # (N, T, H)
#####
#                                     END OF YOUR CODE
#####
return h_all
```

Sanity Check

We can compare the outputs of the two implementations to check if they are consistent.

```
In [ ]: def sanity_check():
    T = 8 # number of time steps
    H = 4 # hidden dimension
    D = 3 # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1
```

```

x = torch.randn(N, T, D)

h_unrolled = unrolled_ssm_forward(W, U, b, x)
h_conv = conv_ssm_forward(W, U, b, x)

diff = (h_unrolled - h_conv).abs().max()
print("Unrolled h(t):")
print(h_unrolled)
print("\nConv-based h(t):")
print(h_conv)
print("\nMax absolute difference:", diff.item())

sanity_check()

```

Implementation Complexity

We can compare the two implementations in terms of efficiency. Particularly, we will compare the time taken by the two implementations to compute the hidden states for a given input and weights with varying number of time steps.

```

In [ ]: import ipywidgets as widgets
        from ipywidgets import interact

def measure_runtime(method_fn, W, U, b, x, warmup=1, repeats=10):
    # Warm-up runs (ignored in timing):
    for _ in range(warmup):
        method_fn(W, U, b, x)

    # Timed runs:
    start = time.time()
    for _ in range(repeats):
        method_fn(W, U, b, x)
    end = time.time()

    avg_time = (end - start) / repeats
    return avg_time

def run():
    T_values_cache = {}
    times_unrolled_vs_T_cache = {}
    times_conv_vs_T_cache = {}

    for H in [4, 8, 16, 32, 64, 128, 256, 512]:
        # We'll keep D, N fixed
        D = 32
        N = 32

        T_values = [8, 32, 128, 256, 512]

```

```

# Build random U, b
U = torch.randn(H, D)*0.1
b = torch.randn(H)*0.1

times_unrolled_vs_T = []
times_conv_vs_T = []

for T in T_values:

    diag_vals = torch.randn(H)*0.05
    W = torch.randn(H, H)*0.05
    x = torch.randn(N, T, D)

    t_unrolled = measure_runtime(unrolled_ssm_forward, W, U, b, x)

    t_conv = measure_runtime(conv_ssm_forward, W, U, b, x)

    times_unrolled_vs_T.append(t_unrolled)
    times_conv_vs_T.append(t_conv)

    T_values_cache[H] = T_values
    times_unrolled_vs_T_cache[H] = times_unrolled_vs_T
    times_conv_vs_T_cache[H] = times_conv_vs_T
return T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache

T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache = run()

@interact(H=widgets.FloatLogSlider(min=2, max=9, base=2, value=4, step=1))
def interactive_benchmark(H):
    """
    Compare unrolled vs. diagonal-convolution RNN forward for various T,
    at a chosen hidden dimension H from the slider.
    """
    H = int(H)
    T_values = T_values_cache[H]
    T_unrolled = times_unrolled_vs_T_cache[H]
    T_conv = times_conv_vs_T_cache[H]

    # Plot
    plt.figure(figsize=(6,4))
    plt.plot(T_values, T_unrolled, label="Unrolled", marker='o')
    plt.plot(T_values, T_conv, label="Conv", marker='s')
    plt.title(f"Runtime vs T, H={H}")
    plt.xlabel("Time Steps (T)")
    plt.ylabel("Runtime (sec)")
    plt.yscale('log')
    plt.grid(True)
    plt.legend()
    plt.show()

```


Question 6

What do you observe about the runtime of the two implementations as T and H increase? Do you observe the same trend as the CPU notebook? Explain your reasoning.

Answer:

Observations:

The answer is obvious, and Claude gets this answer correct. A GPU will obviously result in massive parallelizations and improve the conv efficiency

- **Conv is dramatically faster** (10-100x) than unrolled for most H and T values
- Conv runtime stays nearly flat as T increases; unrolled grows linearly with T
- Even at large H (64, 128), conv remains competitive or faster

This is the **OPPOSITE** trend from CPU:

Aspect	CPU	GPU
Winner for small H	Conv (slightly)	Conv (dramatically)
Winner for large H	Unrolled	Conv (still wins)
Conv scaling with T	Poor ($O(T^2)$)	Nearly flat

Explanation:

1. **GPU parallelism:** Conv1d executes all T output positions in parallel using thousands of CUDA cores. The unrolled loop cannot parallelize across time—each step requires a sequential kernel launch.
2. **Kernel launch overhead:** Each iteration of the unrolled loop launches a separate CUDA kernel ($\sim 10\text{-}100\mu\text{s}$ overhead each). With $T=512$, that's 512 kernel launches dominating compute time.
3. **Conv1d optimization:** cuDNN's conv1d is highly optimized with fused operations, better memory coalescing, and single kernel launch for the entire operation.
4. **Why conv wins on GPU but not CPU:** CPU cannot parallelize conv1d across T effectively (limited cores), while GPU has massive parallelism that conv1d exploits.

Introducing structure: Diagonal Weight Matrices

We can optimize the convolution implementation via adding a constraint on the W matrix ensuring it is diagonal. We can make the convolution implementation more efficient by leveraging depthwise convolutions for implementing the forward operation.

```
In [ ]: def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matrix

    Args:
        W: (H, H) diagonal matrix
        T: (int) number of time steps

    Return:
        kernel of shape (H, 1, T), which can be used in depthwise convolution
    """
    #####
    #                                TODO: Implement the kernel here
    #####
    # Extract diagonal elements: w_diag[i] = W[i, i]
    w_diag = torch.diag(W) # (H,)

    # For diagonal W, W^l = diag(w_diag^l)
    # Kernel[i, 0, l] = w_diag[i]^l for l = 0, 1, ..., T-1
    #
    # Using divide-and-conquer / vectorized approach:
    # powers[l] = w_diag^l, shape (H,)
    # We can compute all powers efficiently using cumulative products or
    # direct exponentiation since it's elementwise

    H = W.shape[0]
    device = W.device
    dtype = W.dtype

    # Create exponents: [0, 1, 2, ..., T-1]
    exponents = torch.arange(T, dtype=dtype, device=device) # (T,)

    # w_diag^l for each l: shape (H, T)
    # w_diag[:, None] has shape (H, 1), exponents has shape (T,)
    # Result: (H, T) where [i, l] = w_diag[i]^l
    powers = w_diag[:, None] ** exponents[None, :] # (H, T)

    # Reshape to (H, 1, T) for depthwise conv
    kernel = powers.unsqueeze(1) # (H, 1, T)
    #####
    #                                END OF YOUR CODE
```

Again, Claude provides highly detailed comments about the math of what is happening in the code. Personally, I think this is super helpful, especially seeing the shapes of the matrices at each line of code.

The staff solution uses a dynamic programming approach, but I think Claude's solution is just as effective, deriving the exponents list first and then elementwise exponentiating to arrive at the kernel

```
#####
return kernel
```

```
In [ ]: def diag_conv_ssm_forward(W, U, b, x):
        """
        Convolution-based forward pass for an RNN with W

        RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
        but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .

        Args:
            W: (H, H) [diagonal in practice]
            U: (H, D)
            b: (H,)
            x: (N, T, D) => N = batch, T = time steps, D = input dim

        Returns:
            h_all: (N, T, H)
        """
        N, T, D = x.shape
        H = W.shape[0]

        s = x @ U.T + b

        s = s.permute(0, 2, 1) # (N,H,T)

        # Build kernel (H,1,T) for depthwise conv
        kernel = make_diag_depthwise_kernel(W, T)
        #####
        #
        #         TODO: Implement the convolution here
        #         Hint: Use `groups` argument
        #####
        # s has shape (N, H, T)
        # kernel has shape (H, 1, T) where kernel[i, 0, l] = w_diag[i]^l
        #
        # For depthwise conv1d:
        #   - groups = H (each channel processed independently)
        #   - input: (N, H, T)
        #   - weight: (H, 1, T) means C_out=H, C_in/groups=1, kernel_size=T
        #
        # Same causal alignment as before: flip kernel and pad left
        kernel_flipped = torch.flip(kernel, dims=[2]) # (H, 1, T)

        # Apply causal padding (T-1 on left, 0 on right)
        s_padded = F.pad(s, (T - 1, 0)) # (N, H, 2T-1)

        # Depthwise convolution with groups=H
        h_conv = F.conv1d(s_padded, kernel_flipped, groups=H) # (N, H, T)

        # Permute back to (N, T, H)
        h_all = h_conv.permute(0, 2, 1) # (N, T, H)
        #####
        #
        #         END OF YOUR CODE
```

Like in the previous notebook, claude again provides a detailed comment section before writing its code (with helpful shape dimensions)

```
#####
return h_all
```

Optimizing the recurrent implementation

Similarly, we can also optimize the recurrent implementation by using the diagonal nature of the W matrix.

```
In [ ]: def diag_unrolled_ssm_forward(W, U, b, x):
        """
        Forward pass for:
            h_{t+1} = W h_t + U x_t + b
        but W is diagonal, i.e. W = diag(diagW).

        Args:
            W: (H, H) diagonal matrix
            U: (H, D)
            b: (H,)
            x: (N, T, D) => batch=N, time steps=T, input dim=D

        Returns:
            h_all: (N, T, H)
        """
        #####
        #                TODO: Implement the optimized recurrent pass here
        #####
        N, T, D = x.shape
        H = W.shape[0]

        # Extract diagonal: w_diag[i] = W[i, i]
        w_diag = torch.diag(W) # (H,)

        # Preallocate output tensor: (N, T, H)
        h_all = torch.zeros(N, T, H, dtype=x.dtype, device=x.device)

        # Initialize hidden state h_0 = 0, shape (N, H)
        h = torch.zeros(N, H, dtype=x.dtype, device=x.device)

        for t in range(T):
            # h_{t+1} = diag(w_diag) @ h_t + U @ x_t + b
            # Since W is diagonal: h_{t+1}[i] = w_diag[i] * h_t[i] + (U @ x_t + b)
            # This is elementwise multiplication instead of matrix multiply
            h = w_diag * h + x[:, t] @ U.T + b # (N, H)
            h_all[:, t, :] = h

        #####
        #                END OF YOUR CODE
        #####
        return h_all
```

Windsurf's solution here is 90% like the staff solution, yet the last part is condensed into 2 lines instead of 3 (more efficient math).

Again I like these detailed comments. Very helpful.

Sanity Check

We can compare the outputs of the two implementations to check if they are consistent similar to above.

```
In [ ]: def diag_sanity_check():
    T = 8    # number of time steps
    H = 4    # hidden dimension
    D = 3    # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.eye(H) * 0.1 ##### WE USE A DIAGONAL MATRIX HERE

    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = diag_unrolled_ssm_forward(W, U, b, x)

    h_conv = diag_conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
    print("\nMax absolute difference:", diff.item())

diag_sanity_check()
```

Measure Runtime with Optimization

Similarly, we will measure performance optimization with the diagonalized implementations.

```
In [ ]: def diag_run():
    T_values_cache = {}
    times_unrolled_vs_T_cache = {}
    times_conv_vs_T_cache = {}

    for H in [4, 8, 16, 32, 64, 128, 256, 512]:
        # We'll keep D, N fixed
        D = 32
        N = 512
```

```

T_values = [8, 32, 128, 256, 512]

# Build random U, b
U = torch.randn(H, D)*0.1
b = torch.randn(H)*0.1

times_unrolled_vs_T = []
times_conv_vs_T = []

for T in T_values:
    diag_vals = torch.randn(H)*0.05

    W = torch.eye(H, H)*0.05 ##### WE USE A DIAGONAL MATRIX HERE

    x = torch.randn(N, T, D)

    t_unrolled = measure_runtime(diag_unrolled_ssm_forward, W, U, b, x)

    t_conv = measure_runtime(diag_conv_ssm_forward, W, U, b, x)

    times_unrolled_vs_T.append(t_unrolled)
    times_conv_vs_T.append(t_conv)

T_values_cache[H] = T_values
times_unrolled_vs_T_cache[H] = times_unrolled_vs_T
times_conv_vs_T_cache[H] = times_conv_vs_T
return T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache

diag_T_values_cache, diag_times_unrolled_vs_T_cache, diag_times_conv_vs_T_cache

@interact(H=widgets.FloatLogSlider(min=2, max=9, base=2, value=4, step=1))
def interactive_benchmark(H):
    """
    Compare unrolled vs. diagonal-convolution RNN forward for various T,
    at a chosen hidden dimension H from the slider.
    """
    H = int(H)
    T_values = diag_T_values_cache[H]
    T_unrolled = diag_times_unrolled_vs_T_cache[H]
    T_conv = diag_times_conv_vs_T_cache[H]

    # Plot
    plt.figure(figsize=(6,4))
    plt.plot(T_values, T_unrolled, label="Unrolled", marker='o')
    plt.plot(T_values, T_conv, label="Diag-Conv", marker='s')
    plt.title(f"Runtime vs T, H={H} diagonal weights")
    plt.xlabel("Time Steps (T)")
    plt.ylabel("Runtime (sec)")
    plt.grid(True)
    plt.legend()
    plt.show()

```

Question 7

What do you observe here? How do your findings different from the unstructured matrix case? Explain your reasoning.

Claude answered this question with the images I provided

Answer:

Observations:

- Depthwise convolution is **overwhelmingly faster** than unrolled—by 50-100x across all H and T values
- **Diag-Conv runtime**: Nearly flat ($\sim 0.001s$) regardless of T or H
- **Unrolled runtime**: Grows linearly with T, reaches $\sim 0.05s$ at $T=512$
- The advantage is consistent across all H values (4 to 512)

Comparison with Unstructured Case:

This table displaying comparisons provides a clear and accurate trend for each case

Aspect	Unstructured (Full Conv)	Diagonal (Depthwise Conv)
Conv advantage	10-100x faster	50-100x faster
Conv scaling with H	Degrades at $H \geq 128$	Stays flat
Stability	Some spikes at large H	Very stable

Explanation:

The explanation, too, is mathematically correct. diagonal conv processing channels independently is a lot more compute efficient than full convolutions

1. **Depthwise conv is much cheaper:**
 - Full conv: kernel $(H, H, T) \rightarrow O(H^2 \cdot T)$ parameters, $O(H^2 \cdot T^2)$ compute
 - Depthwise: kernel $(H, 1, T) \rightarrow O(H \cdot T)$ parameters, $O(H \cdot T)$ compute
 - Removing H^2 eliminates the bottleneck at large H
2. **Perfect GPU utilization:** Depthwise conv with `groups=H` processes each channel independently, mapping perfectly to GPU parallelism.
3. **Trivial kernel construction:** Computing w_diag^l is just elementwise exponentiation $O(H \cdot T)$, versus $O(H^3 \cdot \log T)$ for matrix powers.
4. **Why unrolled doesn't benefit as much:** Even with $O(H)$ per step instead of $O(H^2)$, we still have T sequential kernel launches—the launch overhead dominates.

Summary: The diagonal constraint enables depthwise convolution, which is dramatically more efficient. This is why modern SSMs (like Mamba) use diagonal or

structured state matrices.